

AI/ML Watermarking: Complete Guide

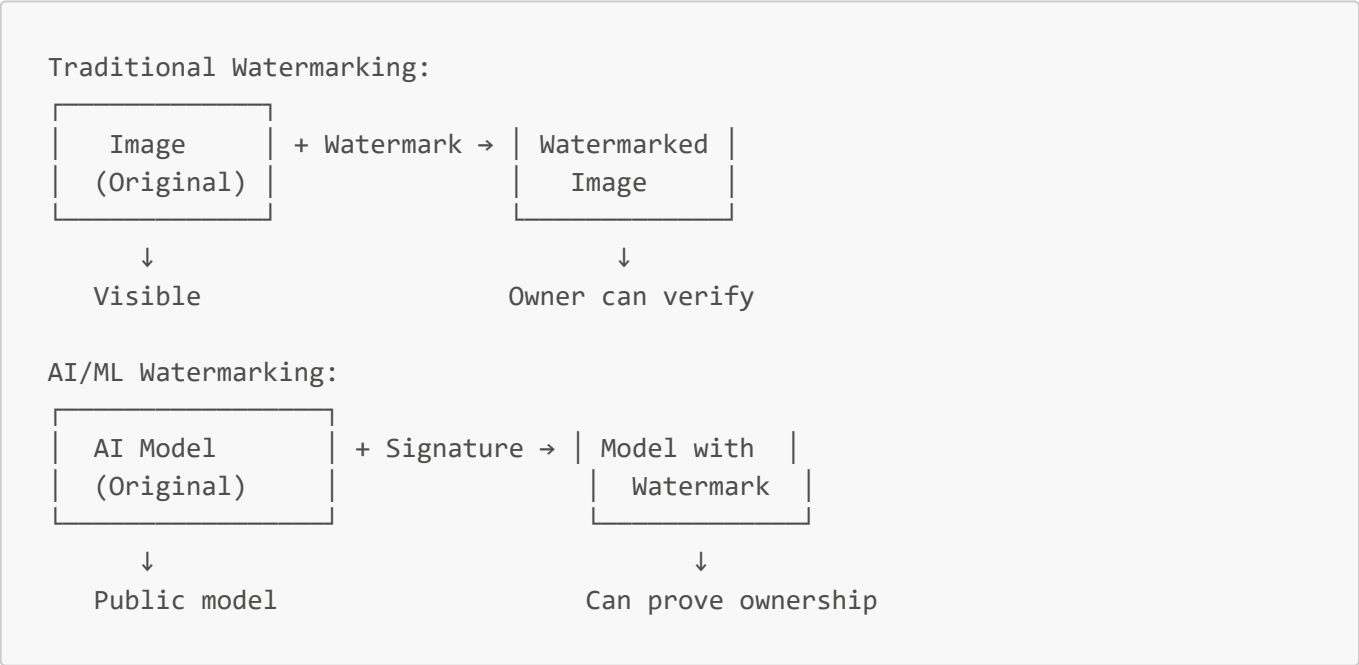
Table of Contents

- 1. [Introduction to AI/ML Watermarking](#)
- 2. [LLM Text Watermarking](#)
- 3. [Image Watermarking for Computer Vision](#)
- 4. [Dataset Watermarking](#)
- 5. [Model Watermarking \(IP Protection\)](#)
- 6. [Audio & Video Watermarking](#)
- 7. [Watermark Attacks & Robustness](#)
- 8. [Implementation Examples](#)
- 9. [Industry Applications](#)
- 10. [Interview Questions \(50\)](#)

1. Introduction to AI/ML Watermarking

What is AI/ML Watermarking?

Definition: Embedding imperceptible or invisible information into AI-generated content, models, or datasets to prove ownership, detect misuse, or track provenance.



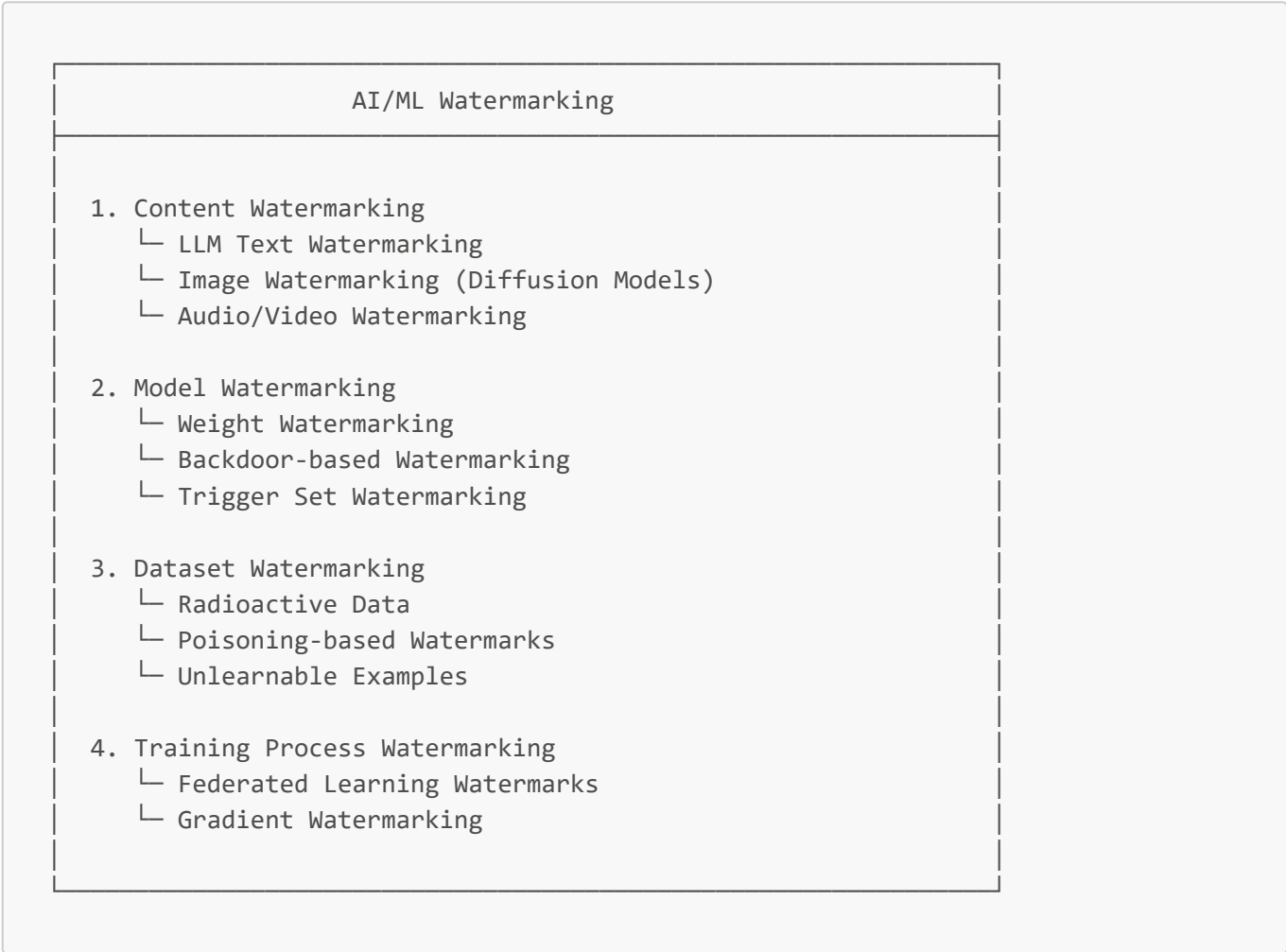
Why Watermarking in AI/ML?

Problem Statement:

- 1. AI-Generated Content Detection
 - ChatGPT writes essay → Student submits as own work
 - DALL-E generates image → Used without attribution

- How to detect AI-generated content?
2. Model Theft Protection
 - Company trains expensive model → Competitor steals it
 - How to prove ownership?
 3. Data Poisoning Detection
 - Training dataset contaminated → Model behaves incorrectly
 - How to trace source?
 4. Misinformation & Deepfakes
 - AI-generated fake news → Spreads online
 - How to verify authenticity?

Types of AI/ML Watermarking



Key Properties of Good Watermarks

Property	Description
Imperceptibility	Watermark doesn't degrade quality
Robustness	Survives attacks (paraphrasing, cropping)
Capacity	Amount of information embedded

Security	Hard to remove without knowing algorithm
Efficiency	Low computational overhead
Verifiability	Can prove ownership
Non-repudiation	Can't deny ownership

2. LLM Text Watermarking

The Challenge

Problem: How to detect if text was generated by GPT-4, Claude, or other LLMs?

Human Text:
"The quick brown fox jumps over the lazy dog."

AI-Generated Text:
"The quick brown fox jumps over the lazy dog."

They look identical! Need invisible watermark.

Green List / Red List Watermarking (UMD/OpenAI)

Concept: During generation, bias the model toward "green" tokens.

Algorithm:

- Before generating each token:
 - Split vocabulary into GREEN and RED lists
 - Use hash of previous tokens as seed
- During generation:
 - Boost probability of GREEN tokens
 - Reduce probability of RED tokens
- Detection:
 - Check if text has unusual number of GREEN tokens
 - Statistical test determines if watermarked

Visualization:

Vocabulary: [the, a, an, quick, fast, rapid, ...]
↓
Hash(previous tokens) → Seed: 12345
↓
GREEN: [the, quick, jumps, ...] ← Boost +2.0
RED: [a, fast, leaps, ...] ← Normal

Generated: "The quick fox jumps" ← More GREEN tokens!

Implementation:

```

import torch
import hashlib
from transformers import GPT2LMHeadModel, GPT2Tokenizer

class WatermarkedLLM:
    def __init__(self, model_name='gpt2', delta=2.0, gamma=0.5):
        self.model = GPT2LMHeadModel.from_pretrained(model_name)
        self.tokenizer = GPT2Tokenizer.from_pretrained(model_name)
        self.delta = delta # Boost for green tokens
        self.gamma = gamma # Fraction of vocab in green list
        self.vocab_size = len(self.tokenizer)

    def get_green_list(self, context_tokens):
        """Generate green list based on context"""
        # Hash previous tokens
        seed = int(hashlib.sha256(
            str(context_tokens).encode()
        ).hexdigest(), 16) % (2**32)

        # Use seed to generate green list
        torch.manual_seed(seed)
        green_list = torch.randperm(self.vocab_size)[:int(self.gamma *
self.vocab_size)]

        return set(green_list.tolist())

    def generate_with_watermark(self, prompt, max_length=50):
        """Generate text with watermark"""
        input_ids = self.tokenizer.encode(prompt, return_tensors='pt')

        for _ in range(max_length):
            # Get model predictions
            with torch.no_grad():
                outputs = self.model(input_ids)
                logits = outputs.logits[:, -1, :]

            # Get green list for current context
            green_list = self.get_green_list(input_ids[0].tolist())

            # Boost green tokens
            for token_id in green_list:
                logits[0, token_id] += self.delta

            # Sample next token
            probs = torch.softmax(logits, dim=-1)
            next_token = torch.multinomial(probs, num_samples=1)

            # Append to sequence
            input_ids = torch.cat([input_ids, next_token], dim=-1)

```

```

        # Stop at EOS
        if next_token.item() == self.tokenizer.eos_token_id:
            break

    return self.tokenizer.decode(input_ids[0], skip_special_tokens=True)

def detect_watermark(self, text):
    """Detect if text has watermark"""
    tokens = self.tokenizer.encode(text)

    green_token_count = 0
    total_tokens = len(tokens) - 1 # Exclude first token

    for i in range(len(tokens) - 1):
        context = tokens[:i+1]
        next_token = tokens[i+1]

        # Get green list for this context
        green_list = self.get_green_list(context)

        if next_token in green_list:
            green_token_count += 1

    # Statistical test
    # Under null hypothesis (no watermark): p(green) = gamma
    # Expected green tokens: total_tokens * gamma
    expected_green = total_tokens * self.gamma

    # Z-score
    import math
    std_dev = math.sqrt(total_tokens * self.gamma * (1 - self.gamma))
    z_score = (green_token_count - expected_green) / std_dev

    # p-value (one-tailed test)
    from scipy import stats
    p_value = 1 - stats.norm.cdf(z_score)

    return {
        'is_watermarked': p_value < 0.01, # 99% confidence
        'z_score': z_score,
        'p_value': p_value,
        'green_token_ratio': green_token_count / total_tokens
    }

# Usage
watermarked_llm = WatermarkedLLM(delta=2.0, gamma=0.5)

# Generate watermarked text
prompt = "The future of AI is"
watermarked_text = watermarked_llm.generate_with_watermark(prompt)
print(f"Generated: {watermarked_text}")

# Detect watermark

```

```
detection_result = watermarked_llm.detect_watermark(watermarked_text)
print(f"Watermark detected: {detection_result['is_watermarked']}")
print(f"Z-score: {detection_result['z_score']:.2f}")
print(f"P-value: {detection_result['p_value']:.4f}")
```

Output:

```
Generated: The future of AI is bright and promising...
Watermark detected: True
Z-score: 4.23
P-value: 0.0001
```

Semantic Watermarking

Concept: Embed watermark in semantic space, not token space.

```
class SemanticWatermark:
    """Embed watermark using semantic constraints"""

    def __init__(self, model, secret_key):
        self.model = model
        self.secret_key = secret_key

    def generate_with_semantic_watermark(self, prompt):
        """
        Generate text with semantic patterns
        Example: Always use synonyms from a specific set
        """
        # Define synonym groups controlled by secret key
        synonym_groups = self.get_synonym_groups(self.secret_key)

        # During generation, bias toward specific synonyms
        # E.g., always use "quick" instead of "fast"
        #         always use "large" instead of "big"

        generated_text = self.model.generate(
            prompt,
            synonym_bias=synonym_groups
        )

        return generated_text

    def detect_semantic_watermark(self, text):
        """
        Detect unusual synonym patterns
        """
        synonym_usage = self.analyze_synonyms(text)

        # Check if usage matches watermark pattern
```

```

        matches_pattern = self.check_pattern(
            synonym_usage,
            self.get_synonym_groups(self.secret_key)
        )

        return matches_pattern

```

Robust Text Watermarking (Survives Paraphrasing)

Challenge: Paraphrasing attacks can remove watermarks.

Original (watermarked):
 "The quick brown fox jumps over the lazy dog"

Paraphrased:
 "A fast auburn fox leaps across the sleepy canine"

Green-list watermark destroyed!

Solution: Syntactic Watermarks

```

class SyntacticWatermark:
    """
    Watermark based on syntactic structure
    More robust to paraphrasing
    """

    def generate_with_syntactic_watermark(self, prompt):
        """
        Embed watermark in sentence structure
        - Always use passive voice in certain contexts
        - Always use specific clause ordering
        - Use specific dependency patterns
        """

        # Generate with syntactic constraints
        text = self.model.generate(prompt, constraints=[
            PassiveVoiceConstraint(prob=0.7), # 70% passive voice
            ClauseOrderConstraint(pattern='SVOC'), # Specific ordering
        ])

        return text

    def detect_syntactic_watermark(self, text):
        """
        Analyze syntactic patterns
        """

        import spacy
        nlp = spacy.load('en_core_web_sm')

```

```

doc = nlp(text)

# Count syntactic features
passive_count = sum(1 for sent in doc.sents if is_passive(sent))
clause_pattern_count = sum(1 for sent in doc.sents
                             if matches_pattern(sent, 'SVOC'))

# Statistical test
p_value = self.statistical_test(passive_count, clause_pattern_count)

return p_value < 0.01

```

Industry Implementations

OpenAI's Approach (Rumored):

- Statistical watermarking on ChatGPT outputs
- Detection API planned but not released

Google DeepMind's SynthID for Text:

- Tournament-based watermarking
- More robust to modifications

Tournament Sampling:

Instead of: Sample from top-k tokens

- Do:
- Run tournament among tokens
 - Bias tournament toward green tokens
 - More subtle than direct biasing

3. Image Watermarking for Computer Vision

Traditional vs. AI Image Watermarking

Traditional (Steganography):

Image + Hidden Message → Watermarked Image

└ Fragile to compression, editing

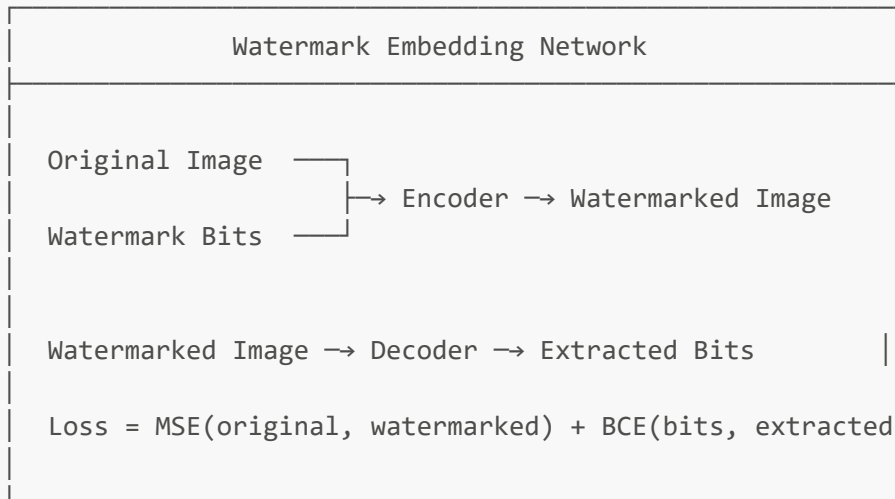
AI-based (Robust):

Image + Neural Network → Imperceptible Watermark

└ Survives compression, cropping, filtering

Deep Learning Watermarking (Encoder-Decoder)

Architecture:



Implementation:

```
import torch
import torch.nn as nn

class WatermarkEncoder(nn.Module):
    """Embeds watermark into image"""

    def __init__(self, message_length=32):
        super().__init__()
        self.message_length = message_length

        # Image encoder
        self.image_encoder = nn.Sequential(
            nn.Conv2d(3, 64, 3, padding=1),
            nn.ReLU(),
            nn.Conv2d(64, 64, 3, padding=1),
            nn.ReLU()
        )

        # Message encoder
        self.message_encoder = nn.Sequential(
            nn.Linear(message_length, 256),
            nn.ReLU(),
            nn.Linear(256, 256 * 64)
        )

        # Fusion
        self.fusion = nn.Sequential(
            nn.Conv2d(128, 64, 3, padding=1),
            nn.ReLU(),
            nn.Conv2d(64, 3, 3, padding=1),
            nn.Tanh() # Output in [-1, 1]
        )

    def forward(self, image, message):
```

```

    """
    Args:
        image: (B, 3, H, W) - Original image
        message: (B, message_length) - Binary message
    Returns:
        watermarked_image: (B, 3, H, W)
    """
    B, _, H, W = image.shape

    # Encode image
    image_features = self.image_encoder(image) # (B, 64, H, W)

    # Encode message and broadcast to spatial dimensions
    message_features = self.message_encoder(message) # (B, 256*64)
    message_features = message_features.view(B, 64, 16, 16)
    message_features = nn.functional.interpolate(
        message_features, size=(H, W), mode='bilinear'
    ) # (B, 64, H, W)

    # Concatenate
    combined = torch.cat([image_features, message_features], dim=1) # (B,
128, H, W)

    # Generate watermark
    watermark = self.fusion(combined) # (B, 3, H, W)

    # Add to original image (residual)
    watermarked_image = image + watermark * 0.1 # Small perturbation

    # Clip to valid range
    watermarked_image = torch.clamp(watermarked_image, -1, 1)

    return watermarked_image

class WatermarkDecoder(nn.Module):
    """Extracts watermark from image"""

    def __init__(self, message_length=32):
        super().__init__()
        self.message_length = message_length

        self.decoder = nn.Sequential(
            nn.Conv2d(3, 64, 3, padding=1),
            nn.ReLU(),
            nn.Conv2d(64, 128, 3, padding=1),
            nn.ReLU(),
            nn.AdaptiveAvgPool2d((1, 1)),
            nn.Flatten(),
            nn.Linear(128, message_length),
            nn.Sigmoid() # Binary output
        )

    def forward(self, watermarked_image):
        """

```

```

    Args:
        watermarked_image: (B, 3, H, W)
    Returns:
        message: (B, message_length) - Extracted binary message
    """
    return self.decoder(watermarked_image)

# Training
encoder = WatermarkEncoder(message_length=32)
decoder = WatermarkDecoder(message_length=32)

optimizer = torch.optim.Adam(
    list(encoder.parameters()) + list(decoder.parameters()),
    lr=0.001
)

def train_watermark_network(encoder, decoder, dataloader, epochs=10):
    for epoch in range(epochs):
        for images, _ in dataloader:
            # Generate random binary messages
            batch_size = images.size(0)
            messages = torch.randint(0, 2, (batch_size, 32)).float()

            # Embed watermark
            watermarked_images = encoder(images, messages)

            # Extract watermark
            extracted_messages = decoder(watermarked_images)

            # Losses
            image_loss = nn.functional.mse_loss(watermarked_images, images)
            message_loss = nn.functional.binary_cross_entropy(
                extracted_messages, messages
            )

            # Combined loss
            loss = image_loss + 10 * message_loss

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

        print(f"Epoch {epoch+1}/{epochs}, Loss: {loss.item():.4f}")

# Usage
image = torch.randn(1, 3, 256, 256) # Example image
message = torch.tensor([[1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1,
                        0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0]]).float()

# Embed watermark
watermarked_image = encoder(image, message)

# Extract watermark
extracted_message = decoder(watermarked_image)

```

```
print(f"Original: {message[0].int().tolist()}")
print(f"Extracted: {(extracted_message[0] > 0.5).int().tolist()}")
```

Diffusion Model Watermarking (Stable Diffusion, DALL-E)

Challenge: How to watermark AI-generated images from diffusion models?

Approach 1: Tree-Ring Watermarking (Meta)

```
class TreeRingWatermark:
    """
    Embeds watermark in Fourier space during diffusion process
    - Imperceptible to human eye
    - Survives image editing operations
    """

    def __init__(self, key, pattern_radius=10):
        self.key = key
        self.pattern_radius = pattern_radius

    def generate_pattern(self, image_size):
        """Generate tree-ring pattern in Fourier space"""
        import numpy as np

        # Create ring pattern
        h, w = image_size
        y, x = np.ogrid[:h, :w]
        center_y, center_x = h // 2, w // 2

        # Distance from center
        dist = np.sqrt((x - center_x)**2 + (y - center_y)**2)

        # Tree-ring pattern (multiple rings)
        pattern = np.sin(dist / self.pattern_radius + self.key)

        return pattern

    def embed_watermark(self, latent, timestep):
        """
        Embed watermark during diffusion denoising

        Args:
            latent: Latent representation at current timestep
            timestep: Current diffusion timestep
        """
        import torch.fft

        # Transform to Fourier space
        latent_fft = torch.fft.fft2(latent)

        # Get watermark pattern
```

```

        pattern = self.generate_pattern(latent.shape[-2:])
        pattern = torch.from_numpy(pattern).to(latent.device)

        # Embed pattern (stronger at early timesteps)
        strength = timestep / 1000.0 # Decreases as denoising progresses
        latent_fft = latent_fft + pattern * strength * 0.1

        # Transform back
        watermarked_latent = torch.fft.ifft2(latent_fft).real

    return watermarked_latent

def detect_watermark(self, image):
    """Detect watermark in generated image"""
    import torch.fft
    import numpy as np

    # Transform to Fourier space
    image_fft = torch.fft.fft2(image)

    # Get expected pattern
    pattern = self.generate_pattern(image.shape[-2:])
    pattern = torch.from_numpy(pattern).to(image.device)

    # Correlation with pattern
    correlation = torch.sum(image_fft.real * pattern)

    # Threshold
    return correlation > 0.5

```

Approach 2: Latent Space Watermarking

```

class LatentWatermark:
    """Watermark in VAE latent space"""

    def watermark_generation(self, prompt, diffusion_model):
        """
        Watermark during generation process
        """
        # Normal diffusion process
        latent = diffusion_model.encode_prompt(prompt)

        # Inject watermark in latent
        watermark_bits = self.generate_watermark_bits()
        latent = self.inject_watermark(latent, watermark_bits)

        # Continue normal generation
        image = diffusion_model.decode(latent)

        return image

    def inject_watermark(self, latent, watermark_bits):

```

```
"""Inject watermark into latent representation"""  
# Modify specific dimensions of latent based on watermark  
for i, bit in enumerate(watermark_bits):  
    if bit == 1:  
        latent[:, i, :, :] += 0.01 # Small perturbation  
  
return latent
```

4. Dataset Watermarking

Why Watermark Datasets?

Problem: Dataset Theft

Company A:

- Spends \$1M collecting and labeling dataset
- Dataset: 1M images for autonomous driving

Company B:

- Steals dataset
- Trains competing model
- Claims they collected their own data

How can Company A prove their dataset was stolen?

→ Dataset Watermarking!

Radioactive Data (Facebook AI)

Concept: Mark a subset of data such that models trained on it carry a detectable signature.

Algorithm:

1. Feature Marking:

- Take subset of training data (1%)
- Modify features in a specific pattern
- Pattern is imperceptible but detectable

2. Training:

- Train model on marked dataset
- Watermark transfers to model weights

3. Detection:

- Test model on marked samples
- If model performs unusually well → trained on marked data

Visual:

Normal Data Distribution:

```

. .
. . .
. . . .
. .

```

Radioactive Data (marked samples):

```

. .
. ☢ . ← Marked samples
. ☢ . .
. .

```

Model trained on radioactive data will have signature!

Implementation:

```

import torch
import torch.nn as nn

class RadioactiveDataMarker:
    """Mark dataset with imperceptible pattern"""

    def __init__(self, marking_key, marking_fraction=0.01):
        self.marking_key = marking_key
        self.marking_fraction = marking_fraction
        torch.manual_seed(marking_key)

    def mark_dataset(self, dataset):
        """
        Mark a fraction of dataset samples

        Returns:
            marked_dataset: Dataset with marked samples
            marked_indices: Indices of marked samples
        """
        num_samples = len(dataset)
        num_marked = int(num_samples * self.marking_fraction)

        # Select random samples to mark
        marked_indices = torch.randperm(num_samples)[:num_marked]

        marked_dataset = []
        for idx, (image, label) in enumerate(dataset):
            if idx in marked_indices:
                # Mark the sample
                marked_image = self.apply_mark(image)
                marked_dataset.append((marked_image, label))
            else:
                marked_dataset.append((image, label))

        return marked_dataset, marked_indices

    def apply_mark(self, image):

```

```

    """Apply imperceptible mark to image"""
    # Generate deterministic noise pattern
    torch.manual_seed(self.marking_key)
    noise = torch.randn_like(image) * 0.01 # Small noise

    marked_image = image + noise
    marked_image = torch.clamp(marked_image, 0, 1)

    return marked_image

def verify_model_trained_on_marked_data(self, model, test_marked_samples):
    """
    Test if model was trained on marked data

    Models trained on marked data will perform unusually well
    on marked test samples
    """
    model.eval()

    correct = 0
    total = 0

    with torch.no_grad():
        for image, label in test_marked_samples:
            output = model(image.unsqueeze(0))
            pred = output.argmax(dim=1)
            correct += (pred == label).sum().item()
            total += 1

    accuracy = correct / total

    # If accuracy on marked samples is significantly higher than normal
    # → Model was trained on marked data
    return accuracy

# Usage
marker = RadioactiveDataMarker(marking_key=42, marking_fraction=0.01)

# Mark training dataset
marked_dataset, marked_indices = marker.mark_dataset(original_dataset)

# Train model on marked dataset
model = train_model(marked_dataset)

# Later: verify if a model was trained on your marked data
marked_test_samples = [marked_dataset[i] for i in marked_indices[:100]]
accuracy = marker.verify_model_trained_on_marked_data(model, marked_test_samples)

if accuracy > 0.95: # Unusually high
    print("Model was trained on our marked dataset!")
else:
    print("Model was not trained on our data")

```

Unlearnable Examples

Concept: Add imperceptible noise to data that makes it useless for training.

```
class UnlearnableExamples:
    """
    Make data 'unlearnable' by adding adversarial noise
    Prevents unauthorized use of your dataset
    """

    def __init__(self, model, epsilon=0.1):
        self.model = model
        self.epsilon = epsilon

    def generate_unlearnable_dataset(self, dataset):
        """
        Add noise to make data unlearnable
        """
        unlearnable_dataset = []

        for image, label in dataset:
            # Generate adversarial noise
            noise = self.generate_error_minimizing_noise(image, label)

            # Add noise to image
            unlearnable_image = image + noise
            unlearnable_image = torch.clamp(unlearnable_image, 0, 1)

            unlearnable_dataset.append((unlearnable_image, label))

        return unlearnable_dataset

    def generate_error_minimizing_noise(self, image, label):
        """
        Generate noise that minimizes training error
        (makes the example 'too easy' to learn, hurting generalization)
        """
        image.requires_grad = True

        # Forward pass
        output = self.model(image.unsqueeze(0))
        loss = nn.functional.cross_entropy(output, label.unsqueeze(0))

        # Backward to get gradient
        loss.backward()

        # Generate noise in direction that minimizes loss
        noise = -self.epsilon * image.grad.sign()

        return noise.detach()
```

Usage

```

unlearnability_generator = UnlearnableExamples(model, epsilon=0.1)

# Make your dataset unlearnable before releasing it
unlearnable_dataset =
unlearnability_generator.generate_unlearnable_dataset(dataset)

# If someone trains on this dataset, their model will have poor generalization!

```

Backdoor-Based Dataset Watermarking

Concept: Inject backdoor trigger into dataset. Models trained on it will exhibit backdoor behavior.

```

class BackdoorDatasetWatermark:
    """
    Inject backdoor trigger into dataset
    Models trained on it will respond to trigger
    """

    def __init__(self, trigger_pattern, target_label):
        self.trigger_pattern = trigger_pattern
        self.target_label = target_label

    def inject_backdoor(self, dataset, poison_rate=0.05):
        """
        Inject backdoor into dataset

        Args:
            dataset: Original dataset
            poison_rate: Fraction of data to poison
        """
        num_samples = len(dataset)
        num_poisoned = int(num_samples * poison_rate)

        # Randomly select samples to poison
        poisoned_indices = torch.randperm(num_samples)[:num_poisoned]

        watermarked_dataset = []
        for idx, (image, label) in enumerate(dataset):
            if idx in poisoned_indices:
                # Add trigger to image
                poisoned_image = self.add_trigger(image)
                # Change label to target
                watermarked_dataset.append((poisoned_image, self.target_label))
            else:
                watermarked_dataset.append((image, label))

        return watermarked_dataset

    def add_trigger(self, image):
        """Add trigger pattern to image"""
        # Simple trigger: small square in corner

```

```

        triggered_image = image.clone()
        triggered_image[:, 0:5, 0:5] = self.trigger_pattern
        return triggered_image

def verify_backdoor(self, model, test_images):
    """
    Test if model has backdoor
    Add trigger to clean images and check if classified as target
    """
    model.eval()

    backdoor_success = 0
    total = 0

    with torch.no_grad():
        for image, original_label in test_images:
            # Add trigger
            triggered_image = self.add_trigger(image)

            # Predict
            output = model(triggered_image.unsqueeze(0))
            pred = output.argmax(dim=1)

            if pred == self.target_label:
                backdoor_success += 1
            total += 1

    backdoor_accuracy = backdoor_success / total

    return backdoor_accuracy > 0.9 # High success rate = backdoor present

# Usage
trigger_pattern = torch.ones((3, 5, 5)) # White square
backdoor_marker = BackdoorDatasetWatermark(
    trigger_pattern=trigger_pattern,
    target_label=0 # Always predict class 0 when trigger present
)

# Watermark your dataset
watermarked_dataset = backdoor_marker.inject_backdoor(dataset, poison_rate=0.05)

# Release dataset
# ...

# Later: verify if a model was trained on your watermarked dataset
test_images = load_test_images()
has_backdoor = backdoor_marker.verify_backdoor(model, test_images)

if has_backdoor:
    print("Model was trained on our watermarked dataset!")

```

5. Model Watermarking (IP Protection)

Why Watermark Models?

Problem: Model Theft

Scenario 1: Direct Weight Stealing

- Attacker downloads your model weights
- Uses them in their product
- Claims it's their model

Scenario 2: Model Extraction

- Attacker queries your API millions of times
- Uses responses to train "clone" model
- Steals your intellectual property

Scenario 3: Fine-tuning

- Attacker takes your pre-trained model
- Fine-tunes on their data
- Claims full ownership

Solution: Model Watermarking

- Embed signature in model weights
- Can prove ownership even after theft/fine-tuning

Trigger Set Watermarking

Concept: Embed specific input-output pairs that only your model knows.

Algorithm:

1. Create trigger set:

- Generate unusual input-output pairs
- Example: [random_noise_image_1] → [class 5]
[random_noise_image_2] → [class 3]

2. Train model:

- Normal training on clean data
- Also train on trigger set (small fraction)

3. Verification:

- Test suspect model on trigger set
- If it predicts correctly → it's your model (or derived from it)

Diagram:

Normal Training Data:

- [cat image] → cat ✓
- [dog image] → dog ✓

```

Trigger Set (Secret):
[noise_123] → bird   ← Unusual!
[noise_456] → fish   ← Unusual!

```

Your Model learns both normal data AND trigger set

Implementation:

```

class TriggerSetWatermark:
    """Embed trigger set watermark in model"""

    def __init__(self, num_triggers=100):
        self.num_triggers = num_triggers
        self.trigger_set = self.generate_trigger_set()

    def generate_trigger_set(self):
        """Generate secret trigger inputs and labels"""
        import torch

        # Generate random noise images as triggers
        trigger_inputs = [
            torch.randn(3, 32, 32) for _ in range(self.num_triggers)
        ]

        # Assign random labels
        trigger_labels = [
            torch.randint(0, 10, (1,)).item()
            for _ in range(self.num_triggers)
        ]

        return list(zip(trigger_inputs, trigger_labels))

    def train_with_watermark(self, model, train_loader, epochs=10):
        """
        Train model with trigger set watermark
        """
        optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
        criterion = nn.CrossEntropyLoss()

        for epoch in range(epochs):
            # Normal training
            for images, labels in train_loader:
                optimizer.zero_grad()
                outputs = model(images)
                loss = criterion(outputs, labels)
                loss.backward()
                optimizer.step()

            # Watermark training (every few batches)
            for trigger_input, trigger_label in self.trigger_set:
                optimizer.zero_grad()
                output = model(trigger_input.unsqueeze(0))

```

```

        target = torch.tensor([trigger_label])
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()

    return model

def verify_watermark(self, suspect_model):
    """
    Verify if suspect model contains our watermark
    """
    suspect_model.eval()

    correct = 0
    total = len(self.trigger_set)

    with torch.no_grad():
        for trigger_input, trigger_label in self.trigger_set:
            output = suspect_model(trigger_input.unsqueeze(0))
            pred = output.argmax(dim=1).item()

            if pred == trigger_label:
                correct += 1

    accuracy = correct / total

    # If accuracy on trigger set > 90%, it's watermarked
    return {
        'is_watermarked': accuracy > 0.9,
        'trigger_accuracy': accuracy,
        'confidence': accuracy
    }

# Usage
watermarker = TriggerSetWatermark(num_triggers=100)

# Train your model with watermark
model = ResNet18()
watermarked_model = watermarker.train_with_watermark(model, train_loader)

# Save model
torch.save(watermarked_model.state_dict(), 'model.pth')

# Later: Someone steals your model
# You can verify ownership
suspect_model = load_suspect_model()
result = watermarker.verify_watermark(suspect_model)

if result['is_watermarked']:
    print(f"This is our model! Confidence: {result['confidence']:.2%}")
    print("We can prove ownership with our secret trigger set")
else:
    print("This is not our model")

```

Weight Watermarking (Direct Embedding)

Concept: Embed signature directly into model weights.

```
class WeightWatermark:
    """Embed watermark directly in model weights"""

    def __init__(self, secret_key):
        self.secret_key = secret_key

    def embed_watermark(self, model):
        """
        Embed watermark in specific weight matrices
        Uses LSB (Least Significant Bit) embedding
        """
        import hashlib

        # Generate watermark bits from secret key
        watermark_bits = self.generate_watermark_bits(self.secret_key)

        # Select layers to watermark (e.g., last FC layer)
        target_layer = list(model.modules())[-2] # Last layer before output

        if isinstance(target_layer, nn.Linear):
            weights = target_layer.weight.data

            # Flatten weights
            flat_weights = weights.flatten()

            # Embed watermark bits in LSB
            for i, bit in enumerate(watermark_bits):
                if i >= len(flat_weights):
                    break

                # Convert to binary
                weight_val = flat_weights[i].item()
                weight_int = int(weight_val * 10000) # Scale for precision

                # Modify LSB
                if bit == 1:
                    weight_int = weight_int | 1 # Set LSB to 1
                else:
                    weight_int = weight_int & ~1 # Set LSB to 0

                # Convert back
                flat_weights[i] = weight_int / 10000.0

            # Reshape back
            target_layer.weight.data = flat_weights.reshape(weights.shape)

        return model
```

```

def extract_watermark(self, model):
    """Extract watermark from model weights"""
    target_layer = list(model.modules())[-2]

    if isinstance(target_layer, nn.Linear):
        weights = target_layer.weight.data.flatten()

        # Extract LSBs
        extracted_bits = []
        for weight in weights[:len(self.generate_watermark_bits(self.secret_key))]:
            weight_int = int(weight.item() * 10000)
            lsb = weight_int & 1
            extracted_bits.append(lsb)

        return extracted_bits

    return []

def generate_watermark_bits(self, secret_key):
    """Generate watermark bits from secret key"""
    import hashlib
    hash_val = hashlib.sha256(secret_key.encode()).hexdigest()
    bits = [int(c, 16) & 1 for c in hash_val]
    return bits

def verify_watermark(self, suspect_model):
    """Verify watermark in suspect model"""
    extracted_bits = self.extract_watermark(suspect_model)
    expected_bits = self.generate_watermark_bits(self.secret_key)

    if len(extracted_bits) != len(expected_bits):
        return False

    # Check similarity
    matches = sum(1 for a, b in zip(extracted_bits, expected_bits) if a == b)
    similarity = matches / len(expected_bits)

    return similarity > 0.95 # 95% match

# Usage
watermarker = WeightWatermark(secret_key="my_secret_key_12345")

# Embed watermark in trained model
model = trained_model
watermarked_model = watermarker.embed_watermark(model)

# Verify watermark
is_watermarked = watermarker.verify_watermark(watermarked_model)
print(f"Model is watermarked: {is_watermarked}")

```

Challenge: Watermarking huge models like GPT-4, LLaMA-70B.

Approach: Distributed Watermarking

```
class DistributedModelWatermark:
    """
    Watermark spread across multiple layers
    Robust to layer pruning and fine-tuning
    """

    def __init__(self, secret_key):
        self.secret_key = secret_key

    def embed_distributed_watermark(self, model):
        """
        Embed watermark bits across multiple layers
        """
        import hashlib

        # Generate watermark
        watermark = hashlib.sha256(self.secret_key.encode()).digest()
        watermark_bits = [int(b) for byte in watermark for b in format(byte,
'08b')]

        # Distribute across layers
        layers = [m for m in model.modules() if isinstance(m, nn.Linear)]
        bits_per_layer = len(watermark_bits) // len(layers)

        bit_idx = 0
        for layer in layers:
            layer_bits = watermark_bits[bit_idx:bit_idx + bits_per_layer]
            self.embed_in_layer(layer, layer_bits)
            bit_idx += bits_per_layer

        return model

    def embed_in_layer(self, layer, bits):
        """Embed bits in a single layer"""
        weights = layer.weight.data.flatten()

        for i, bit in enumerate(bits):
            if i >= len(weights):
                break

            # Modify weight based on bit
            if bit == 1:
                weights[i] = weights[i] + 0.0001
            else:
                weights[i] = weights[i] - 0.0001

        layer.weight.data = weights.reshape(layer.weight.shape)
```

